" OrbitMind AI– Intelligent Productivity & Study Companion"

by

Md. Imroz Newaz

CSE 11220321052

Zeniya Naznin

CSE 11230121138

Fatema Parvin Kanta

CSE 11230121192

Md. Mahfuzur Rahman Hridoy

CSE 11230121186

WEEK 05 ASSIGNMENT

Course Title: Software Development III: AI-Based Software Project Development

Course Code: CSE 4104

Section: 7D | Spring 2026

Submitted to

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering

Northern University of Business and Technology, Khulna

Khulna 9100, Bangladesh

CONTENTS

CHAPTER I

1.1 Team Information Sheet

Field	Details
Team Name	CSE4104-7D-T04
Section	7D
Project Title	OrbitMind AI — Smart Productivity & Study Assistant
Team Leader	Md. Imroz Newaz (ID: 11220321052)

1.1.1 Team Members

Member Name	Student ID	Role
Md. Imroz Newaz	11220321052	Team Leader & Database Manager
Zeniya Naznin	11230121138	Frontend Developer
Fatema Parvin Kanta	11230121192	Backend Developer
Md. Mahfuzur Rahman Hridoy	11230121186	AI Integration Lead

CHAPTER II

2.1 Project Information

Overview: OrbitMind AI is a comprehensive productivity web application designed to alleviate the academic pressures faced by university students. The system seamlessly integrates standard task management—such as deadline tracking and workload visualization—with advanced Artificial Intelligence powered by Google Gemini. Our platform acts as a proactive academic assistant: it automates the creation of day-by-day study roadmaps, summarizes dense lecture transcripts into easily digestible concepts, and features a conversational AI coach to guide students through complex topics. By finalizing our UI/UX designs, we are ensuring that this powerful functionality is delivered through an accessible, highly intuitive, and distraction-free interface, which is critical for maintaining student focus and reducing cognitive overload.

2.2 Development Roadmap

To ensure a smooth transition from the design phase into full system implementation, our team has established the following agile development roadmap for the remainder of the semester.

- **Week 06: Core Backend & Database Initialization**
    - Set up the Node.js/Express.js server architecture.
    - Initialize the MongoDB Atlas database and define Mongoose schemas (User, Task, Study Plan).
    - Develop the JWT-based authentication APIs (Register/Login).

- **Week 07: Core Frontend & State Management Development**
    - Connect the existing Vanilla HTML/CSS/JS frontend to the backend APIs.
    - Implement client-side session management and route protection.
    - Finalize the dynamic rendering of the user Dashboard and Task Board.

- **Week 08: AI Services Integration**
    - Securely connect the backend to the Google Gemini API.
    - Develop the prompt engineering logic for the AI Study Planner and AI Summarizer.
    - Connect the frontend Chatbot UI to the backend AI streaming routes.

- **Week 09: Advanced Feature Completion**
    - Implement the Notification system and workload analytics charts.
    - Finalize data validation and error handling across all forms.
    - Ensure the UI is fully responsive across mobile and desktop breakpoints.

- **Week 10: System Testing and Debugging**

    - Conduct rigorous unit testing on backend API routes using Postman.
    - Perform End-to-End (E2E) UI testing to ensure user flows operate flawlessly.
    - Resolve any integration bugs between the database and the AI module.

- **Week 11: Final Deployment & Presentation Prep**
    - Deploy the Node.js backend and MongoDB database to a cloud provider (e.g., Render).
    - Host the static frontend interface.
    - Conduct final user acceptance testing and prepare the project defense presentation.

2.3 Team Task Distribution

To ensure strict accountability, efficient parallel development, and high-quality deliverables, project responsibilities have been strategically divided among team members based on their technical strengths.

- **Md. Imroz Newaz (Team Leader, Database Management & Documentation):**
    - Facilitates team coordination, equitable task distribution, progress management, weekly reporting, and acts as the primary point of communication with the instructor.
    - Oversees the entire agile project lifecycle and timeline management.
    - Leads the creation of all formal project documentation (Team Information & Initial Project Idea Selection, Project Proposal, SRS, System Design, UI/UX Planning) and ensures submission compliance.
    - Designs and manages the MongoDB Atlas database architecture and Mongoose schemas.
    - Handles the final cloud deployment and ensures the UI design accurately matches the core SRS requirements.

- **Zeniya Naznin (Frontend Development & UI Testing):**

- Responsible for translating the UI/UX wireframes into functional, responsive HTML5 and CSS3.
- Manages complex DOM manipulation and micro-animations using Vanilla JavaScript.
- Implements client-side form validation and interactive dashboard components.
- Conducts extensive Frontend Testing to ensure cross-device responsiveness and accessibility.

- **Fatema Parvin Kanta (Backend Development & API Testing):**
  - Handles the core Node.js and Express.js server logic.
  - Responsible for building secure RESTful APIs and protecting routes with JWT middleware.
  - Ensures architectural consistency and smooth data flow to the frontend UI.
  - Performs rigorous Backend Testing using Postman to validate API endpoints and server security.

- **Md. Mahfuzur Rahman Hridoy (AI Integration Lead):**
  - Dedicated to the seamless integration of the Google Gemini API.
  - Responsible for writing complex prompt engineering logic and parsing JSON responses from the LLM.
  - Conducts rigorous quality assurance testing on the AI Chatbot and AI Note Summarizer features.
  - Manages the team's GitHub repository, resolves branch conflicts, and maintains the README.md setup instructions.

2.4 Final Technology Setup Plan

The following technology stack has been finalized to meet the rigorous performance, scalability, and security requirements outlined in our System Design document. Each technology was selected specifically to support our AI-driven architecture.

- **Frontend (Client Interface):**
  - **Stack:** HTML5, CSS3, and Vanilla JavaScript (ES6+).
  - **Justification:** We opted for a lightweight, dependency-free Vanilla stack to ensure maximum page loading speeds, pristine CSS theme control, and instant DOM updates without the overhead of heavy virtual DOM frameworks like React.

- **Backend (Application Logic):**
  - **Stack:** Node.js and Express.js.
  - **Justification:** Node.js provides a highly scalable, non-blocking, asynchronous event-driven environment. This is absolutely critical for our system, as waiting for external AI API responses (like Google Gemini) requires asynchronous handling to prevent server freezing.

- **Database (Persistent Storage):**
  - **Stack:** MongoDB Atlas (NoSQL) accessed via Mongoose ODM.
  - **Justification:** Chosen for its flexible, document-based JSON schema. Because our AI Study Planner generates highly complex, nested daily schedules, a NoSQL database allows us to store these dynamic AI responses natively without rigid relational table constraints.

- **AI Services (Generative Engine):**
  - **Stack:** Google Gemini API (via @google/generative-ai SDK).
  - **Justification:** Selected over standard OpenAI models due to its rapid processing speed, generous context window (crucial for summarizing massive lecture transcripts), and superior natural language understanding for academic coaching.

- **Deployment (Production Hosting):**
  - **Stack:** Render (Backend API) and Vercel/Netlify (Static Frontend).

- **Justification:** Provides seamless continuous integration (CI/CD) directly from our GitHub repository, ensuring that any pushed code is automatically deployed to production.

CHAPTER III

3.1 Visual Diagrams

3.1.1 User Flow Diagram

Theoretical Description: The User Flow Diagram visually maps the navigational journey a student takes through the OrbitMind AI platform. It emphasizes a frictionless UX, starting from initial registration/login, progressing to the central Dashboard hub, and branching out into our core functional modules. The flow demonstrates that the user can seamlessly transition between managing manual tasks and triggering advanced AI features (like the Study Planner or AI Coach) without losing context, eventually safely logging out of the system.

Figure 3.1.1: User Flow Diagram

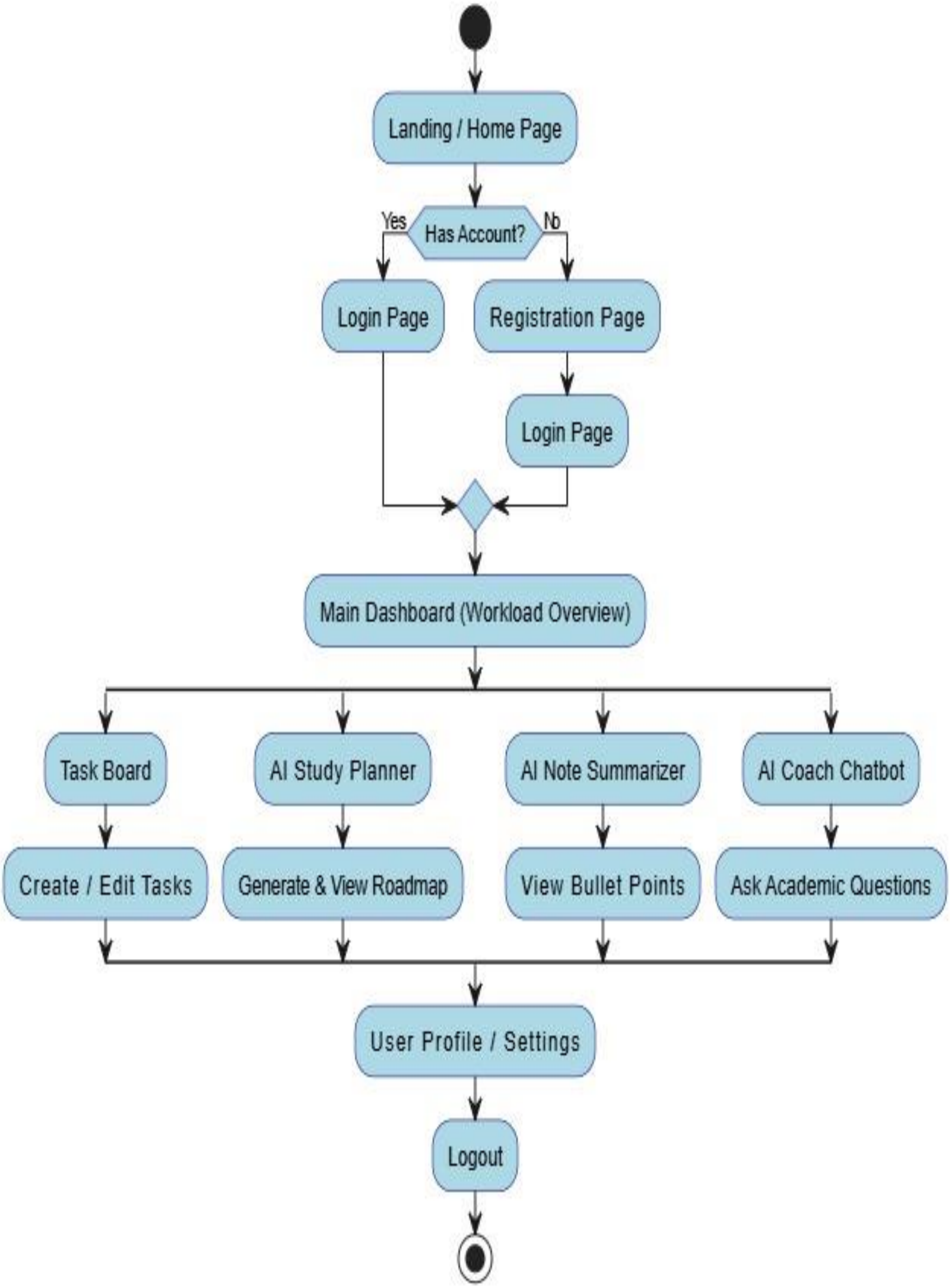

Figure 3.1.1: User Flow Diagram

3.2 UI/Wireframe Screenshots (High-Fidelity)

3.2.1 Landing Page

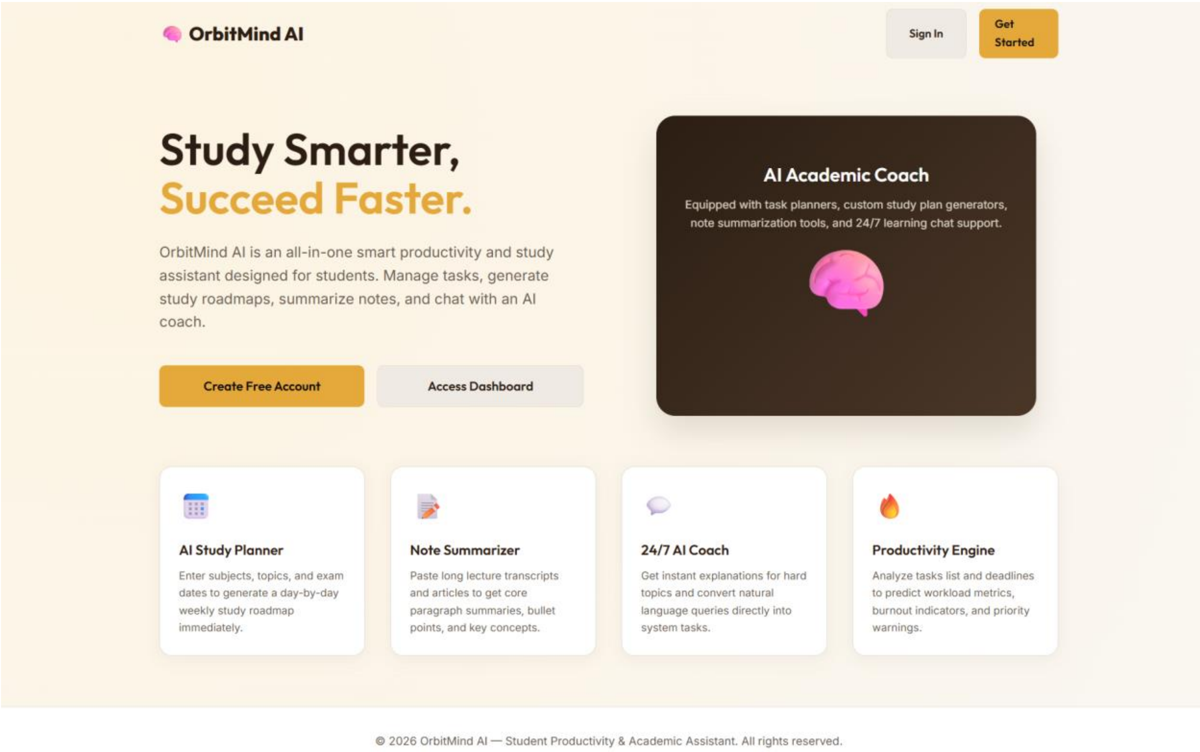

Figure 3.2.1.1: The OrbitMind AI Landing Page.

UX Design Justification: The landing page is designed with a minimalist aesthetic to immediately draw the user's attention to the core value proposition. A prominent Call-To-Action (CTA) button ensures a frictionless entry point for new users, reducing bounce rates and encouraging immediate onboarding.

3.2.2 User Authentication (Registration & Login)

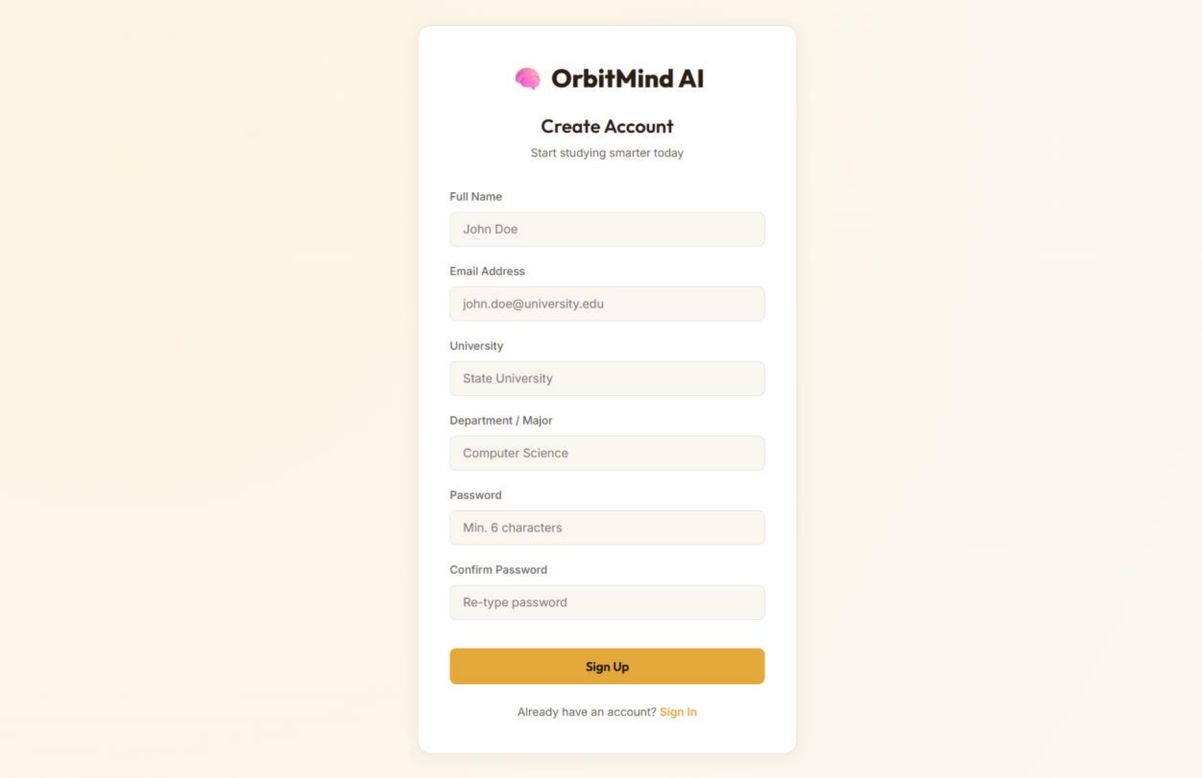

Figure 3.2.2.1: The Registration Interface.

UX Design Justification: Form fields are kept clean and clearly labeled to minimize cognitive load during sign-up. Real-time validation feedback will be implemented here to prevent user frustration before form submission.

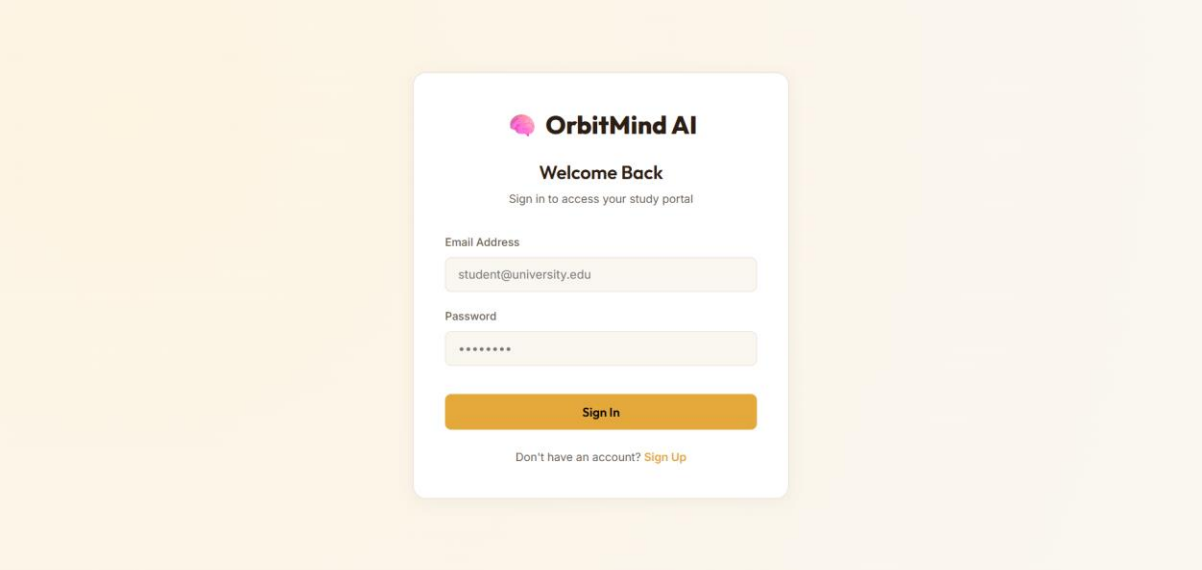

Figure 3.2.2.2: The secure Login Interface.

UX Design Justification: The login screen prioritizes speed and security. By isolating the login form in the center of the screen, we eliminate visual distractions, allowing the student to authenticate and access their dashboard as quickly as possible.

3.2.3 Central Dashboard

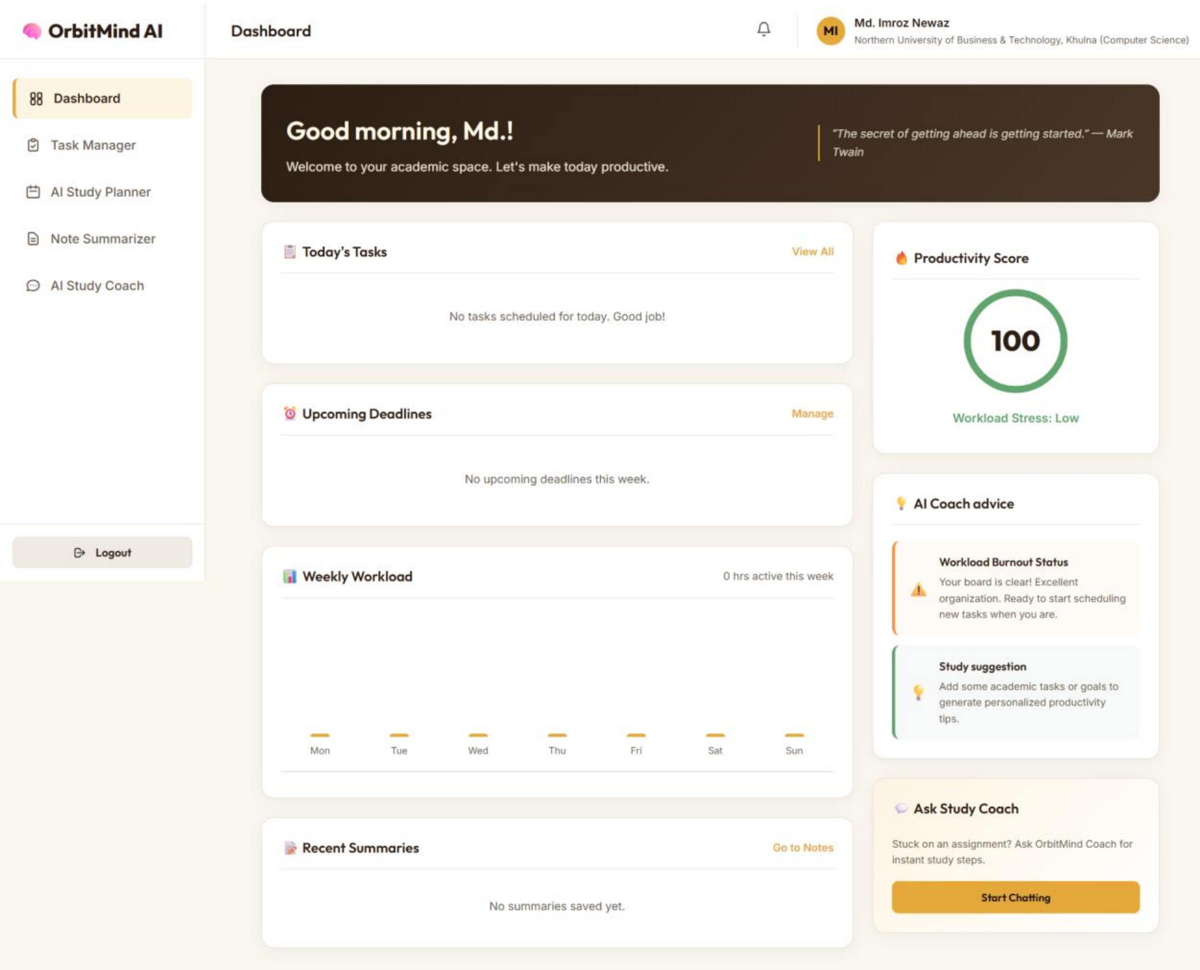

Figure 3.2.3.1: The Main Dashboard Interface.

UX Design Justification: As the central hub of the application, the dashboard utilizes a modular card-based layout. This allows students to process complex information (such as upcoming deadlines and AI recommendations) at a glance without feeling overwhelmed by dense text.

3.2.4 Task Management

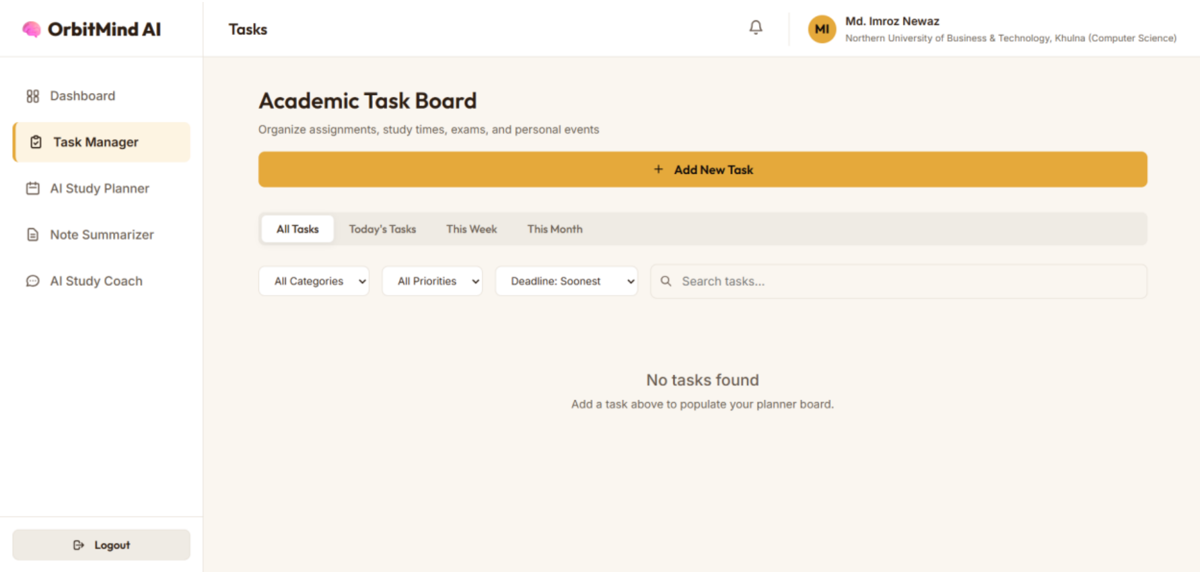

Figure 3.2.4.1: The Task Board Interface.

UX Design Justification: The task management interface focuses on hierarchy and organization. Tasks are visually separated by priority and status, utilizing subtle color coding so students can instantly identify urgent assignments and track their progress efficiently.

3.2.5 AI Interaction Pages

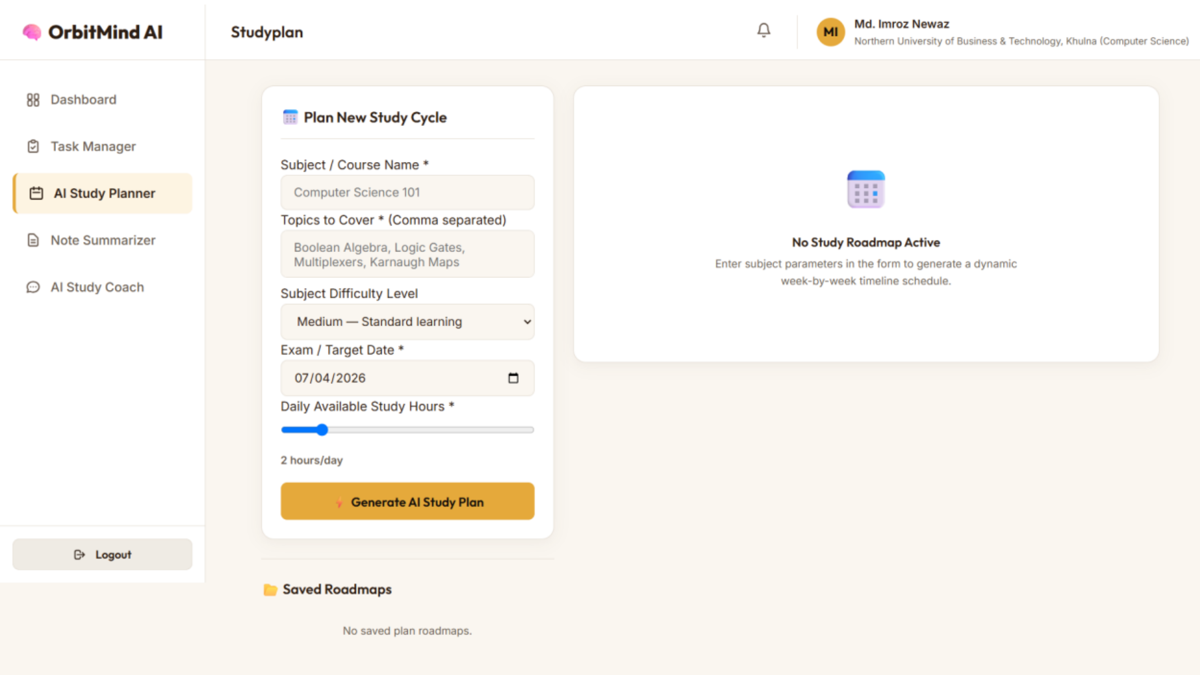

Figure 3.2.5.1: The AI Study Planner Interface.

UX Design Justification: Generating a study roadmap requires complex user input (syllabus, dates, hours). The UI simplifies this by using step-by-step input fields. The resulting AIgenerated plan is rendered in a highly structured, day-by-day timeline format for maximum readability.

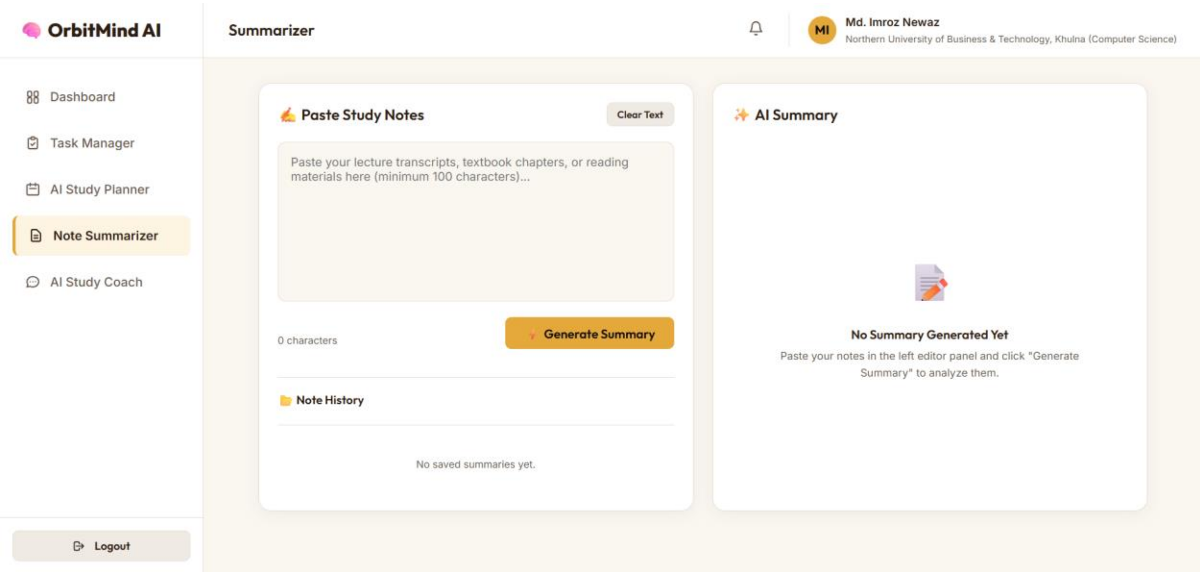

Figure 3.2.5.2: The AI Note Summarizer Interface.

UX Design Justification: This interface features a split-pane or clearly delineated layout. Students paste massive walls of text on one side and receive concise, visually distinct bullet points on the other, creating an immediate and satisfying contrast that emphasizes the AI's value.

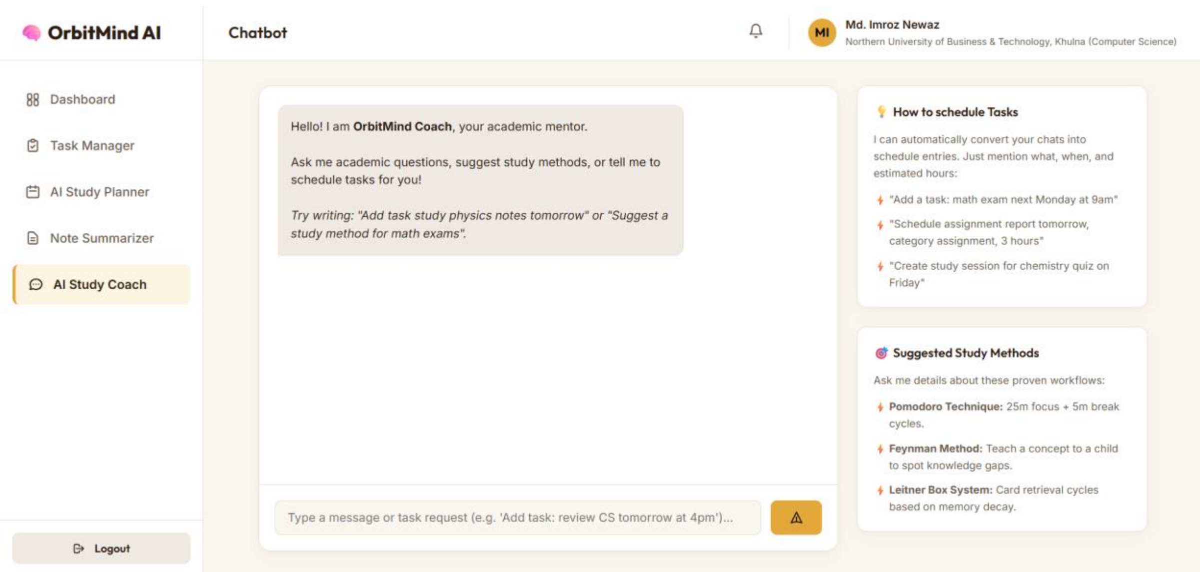

Figure 3.2.5.3: The AI Coach Chatbot Interface.

UX Design Justification: Designed to mimic familiar messaging applications, this interface leverages established UX patterns. The conversational UI lowers the learning curve, making the AI feel approachable, responsive, and easy to interact with during late-night study sessions.